Computational Frameworks

MapReduce

OUTLINE

1. MapReduce
2. Basic Techniques

MapReduce

Motivating scenario

At the dawn of the big-data era (early 2000's), the following scenario was rapidly emerging

- In a wide spectrum of domains there was an increasing need to analyze large amounts of data.
- Available tools and commonly used platforms could not practically handle very large datasets.
  - Algorithms with high polynomial complexities were unfeasible, and platforms had limited main memory.
  - In extreme cases, even *touching all data items* would prove quite time consuming. (E.g., $\simeq 50 \cdot 10^9$ *web pages of about 20KB each* $\rightarrow$ *1000TB of data*.)
- The use of powerful computing systems, which has been confined until then to a limited number of applications – typically from computational sciences (*physics, biology, weather forecast, simulations*) – was becoming necessary for a much wider array of applications.

e.g. SUPERCOMPUTERS

Motivating scenario

Powerful computing systems, which feature multiple processors, multiple storage devices, and high-speed communication networks, pose several problems

- They are costly to buy and to maintain, and they become rapidly obsolete.
- Fault-tolerance becomes serious issue: a large number of components implies a low *Mean-Time Between Failures* (MTBF).
- Developing software that effectively benefits from parallelism requires *sophisticated programming skills*.

System with N components (independent w.r.t. failure)

$p = Pr(\text{a given component fails in 1 time unit})$

$\Rightarrow$ mean time between failures (1 component) $= \frac{1}{p}$

$Pr(\text{one of the components fails in 1 t.u}) = 1-(1-p)^N$

$\Rightarrow$ mean time between failures (for N components)

$$= \frac{1}{1-(1-p)^N} \xrightarrow[N \to +\infty]{} 1$$

MapReduce

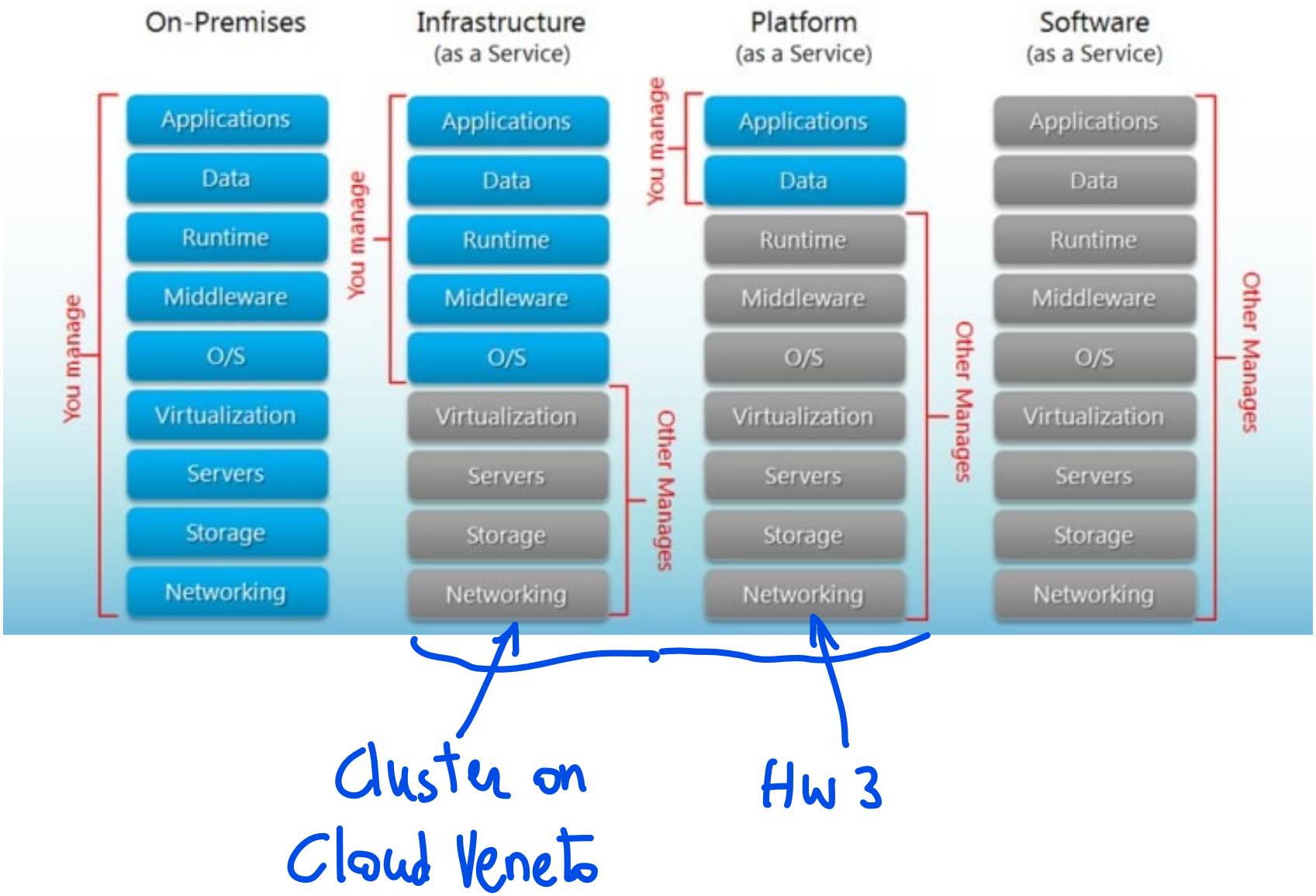
- Introduced by Google in 2004 as a programming framework for big data processing on distributed platforms.
- Its original formulation (see [DG08]) contained
  - A programming model
  - An implementation tailored towards a cluster-based computing environment.
- Since then, MapReduce implementations have been employed on clusters of commodity processors and cloud infrastructures for a wide array of big-data applications
- **Main features:**
  - Data centric view
  - Inspired by functional programming (map, reduce functions)
  - Ease of algorithm/program development. Messy details (e.g., task allocation; data distribution; fault-tolerance; load-balancing) are hidden to the programmer

Platforms

Typically, MapReduce applications run on

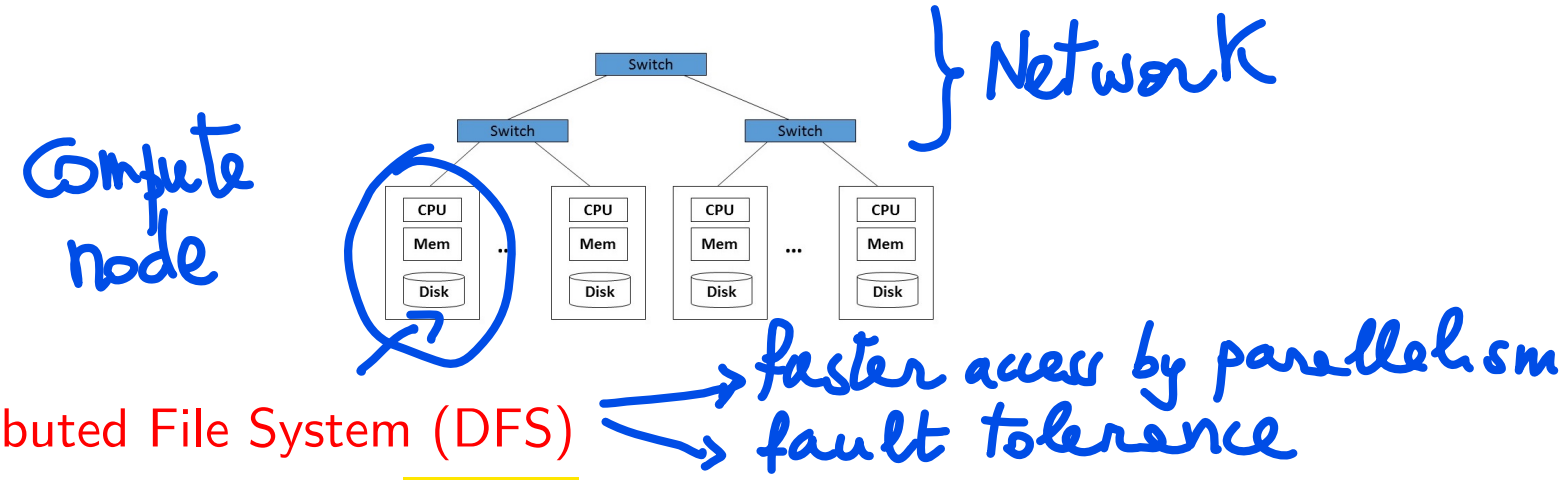
- Clusters of commodity processors (On-premises)
- Platforms from cloud providers (Amazon AWS, Microsoft Azure)
  - IaaS (*Infrastructure as a Service*): provides the users computing infrastructure and physical (or virtual) machines. This is the case of CloudVeneto which provides us a cluster of virtual machines.
  - PaaS (*Platform as a Service*): provides the users computing platforms with OS; execution environments, etc.

Platforms


On-Premises
Infrastructure (as a Service)
Platform (as a Service)
Software (as a Service)
Applications
Data
Runtime
Middleware
O/S
Virtualization
Servers
Storage
Networking
You manage
Other Manages
Cluster on Cloud Veneto
Hw 3

Typical cluster architecture

- Racks of 16-64 compute nodes (commodity hardware), connected (within each rack and among racks) by fast switches (e.g., 10 Gbps Ethernet)



- Distributed File System (DFS)
  - Files divided into *chunks* (e.g., 64MB per chunk)
  - Each chunk replicated (e.g., 2x o 3x) with replicas in different nodes and, possibly, in different racks
  - The distribution of the chunks of a file is represented into a *master node* file which is also replicated. A directory (also replicated) records where all master nodes are.
  - Examples: Google File System (GFS); Hadoop Distributed File System (HDFS)

MapReduce-Hadoop-Spark

Several software frameworks have been proposed to support MapReduce programming. For example:

- Apache Hadoop: 

most popular MapReduce implementation (*often used as synonymous of MapReduce*) from which an entire ecosystem of alternatives has stemmed, aimed at improving its (initially) very poor performance. The Hadoop Distributed File System is still widely used.

- Apache Spark (*learned in this course*): one of the most popular and widely used frameworks for big-data applications. It supports the implementation of MapReduce computations, but provides a much richer programming environment. It uses the HDFS.

In this course

* MapReduce: Conceptual model for high-level algorithm design/analysis
* Spark: Programming framework for algorithm implementation

MapReduce computation

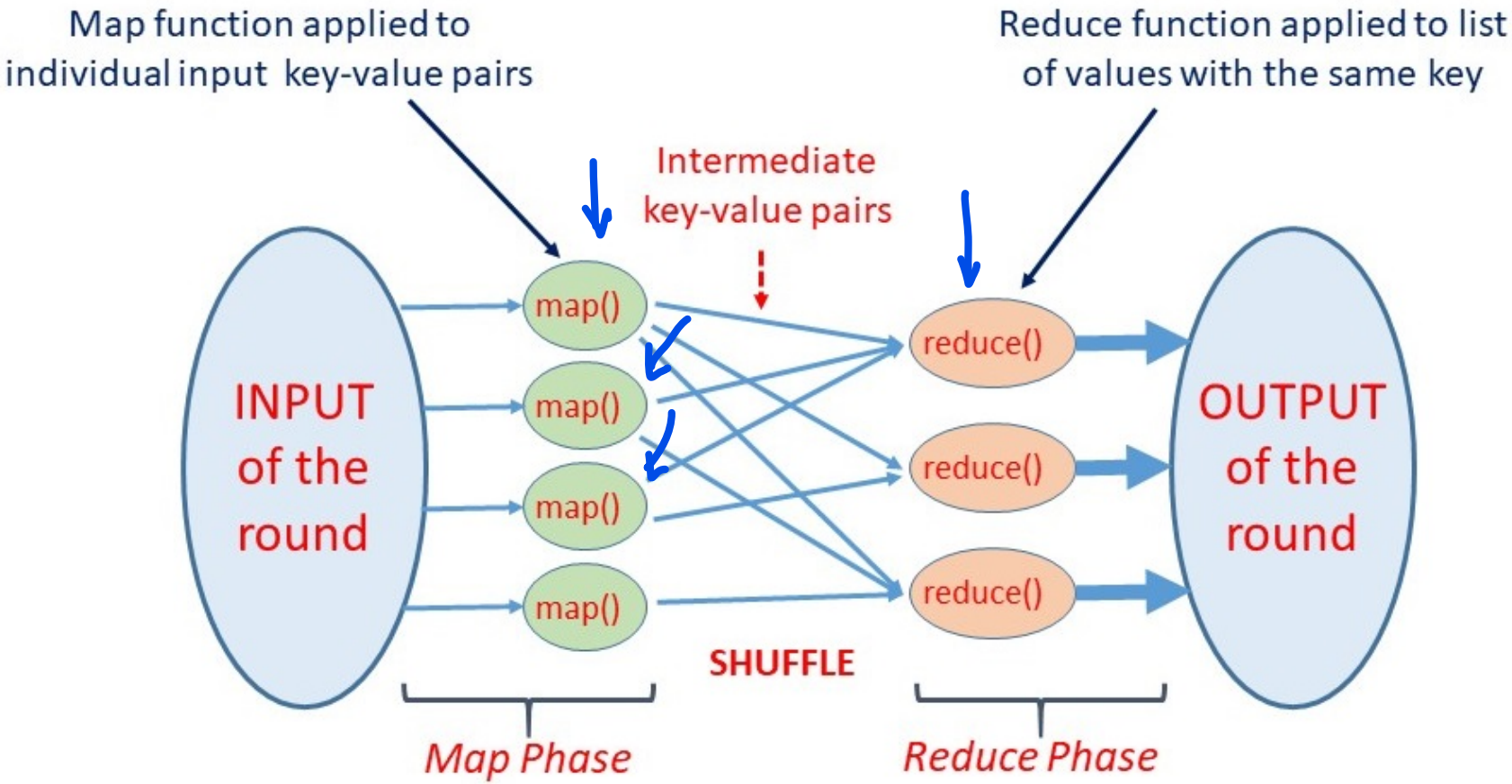
A MapReduce computation can be viewed as a sequence of rounds.

A round transforms a set of key-value pairs into another set of key-value pairs (*data centric view* !), through the following two phases

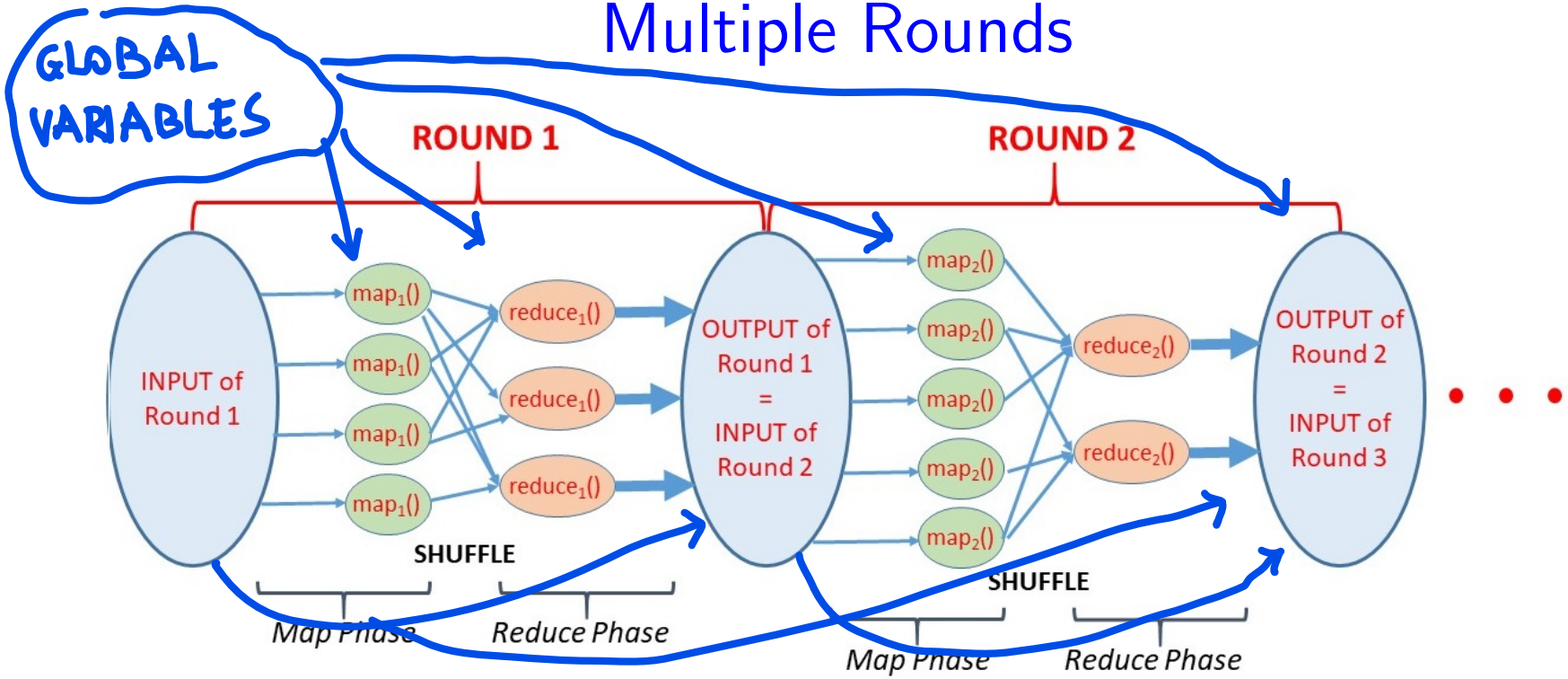
- Map phase: a user-specified map function is applied separately to each input key-value pair and produces ≥ 0 other key-value pairs, referred to as intermediate key-value pairs.
- Reduce phase: the intermediate key-value pairs are grouped by key, and for each key k separately, a user-specified reduce function is applied to the list of values of the intermediate pairs with key k, producing ≥ 0 key-value pairs, which is the output of the round.

Remark: between the two phases, a SHUFFLE of the intermediate key-value pairs is needed.

One Round


Map function applied to individual input key-value pairs
Reduce function applied to list of values with the same key
Intermediate key-value pairs
INPUT of the round
map()
map()
map()
map()
reduce()
reduce()
reduce()
OUTPUT of the round
SHUFFLE
Map Phase
Reduce Phase

Multiple Rounds



In practice, a bit more flexibility is used:

- the input of a round may comprise a subset of the initial input data and/or subsets of the output of previous rounds, if any.
- some data can be used as global variables known to all processing elements that carry out the computation. Programming frameworks such as Spark make provisions in this sense.

Why key-value pairs?

The use of key-value pairs in MapReduce seems an unnecessary complication, and one might be tempted to regard individual data items simply as *objects* belonging to some domain, without imposing a distinction between keys and values.

Justification. MapReduce is data-centric and focuses on data trasformations which are independent of where data actually reside. The keys are needed as addresses to reach the objects, and as labels to define the groups in the reduce phases.

Practical considerations.

- One should choose the key-value representations in the most convenient way. The domains for key and values may change from one round to another.
- Programming frameworks such as Spark, while containing explicit provisions for handling key-value pairs, allow also to manage data without the use of keys. In this case, internal addresses/labels, not explicitly visible to the programmer, are used.

Specification of a MapReduce algorithm

A *MapReduce (MR) algorithm* should be specified so that

- The input and output of the algorithm is clearly defined
- The sequence of rounds executed for any given input instance is unambiguously implied by the specification
- For each round the following aspects are clear
  - input, intermediate and output sets of key-value pairs
  - functions applied in the map and reduce phases.
- Meaningful values (or asymptotic) bounds for the key performance indicators (defined later) can be derived.

Pseudocode style

To specify a MR algorithm with a fixed number of rounds R, we will use the following style:

Input: description of the input as set of key-value pairs
Output: description of the output as set of key-value pairs

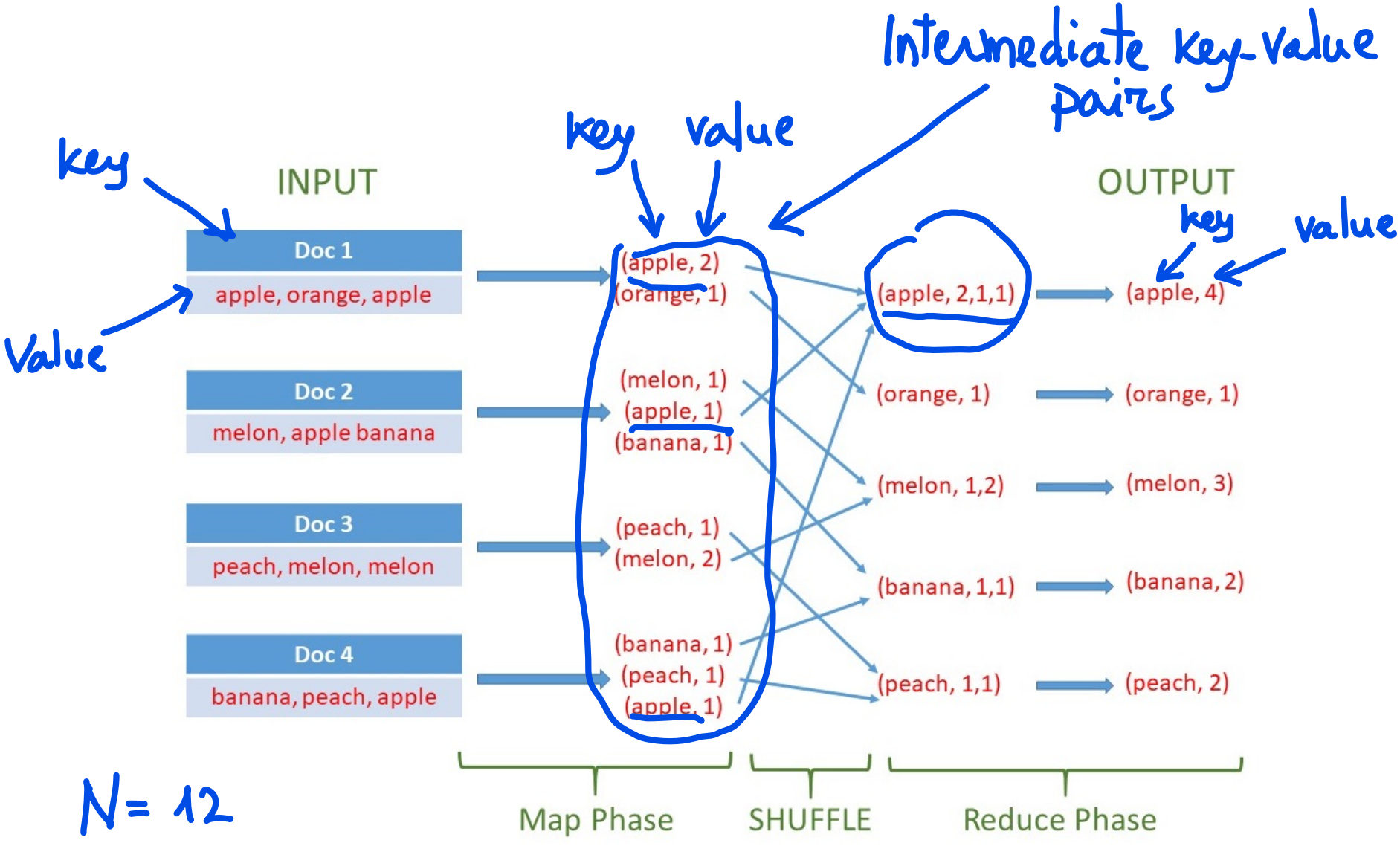
Round 1:

- *Map phase*: description of the function applied to each key-value pair
- *Reduce phase*: description of the function applied to each ~~group of key-value pairs with the same key~~ key-listOfValues pair

Round 2, 3, ..., R: similarly

Observation: *for simplicity, we sometime provide a high-level description of a round, which, however, must enable a straightforward yet tedious derivation of the Map and Reduce phases.*

Example: Word count



N = 12

Example: Word count

Input : Set of documents $D_1, D_2, \ldots, D_k$

D_i = (name, list-of-words)

→ with repetitions

N = total number of word occurrences across all docs

Output: $\{(w, c(w)) \cdot w \in D_1, D_2, \ldots, D_k$

$c(w)$ = # occurrences of w in $D_1, D_2, \ldots, D_k$

Example: Word count

Round 1

* $\forall\ 1 \le i \le k$ separately

$$D_i \longrightarrow \{(w, c_i(w)) \quad w \in D_i,\ c_i(w) = \#\text{occurrences of } w \text{ in } D_i\}$$

map function

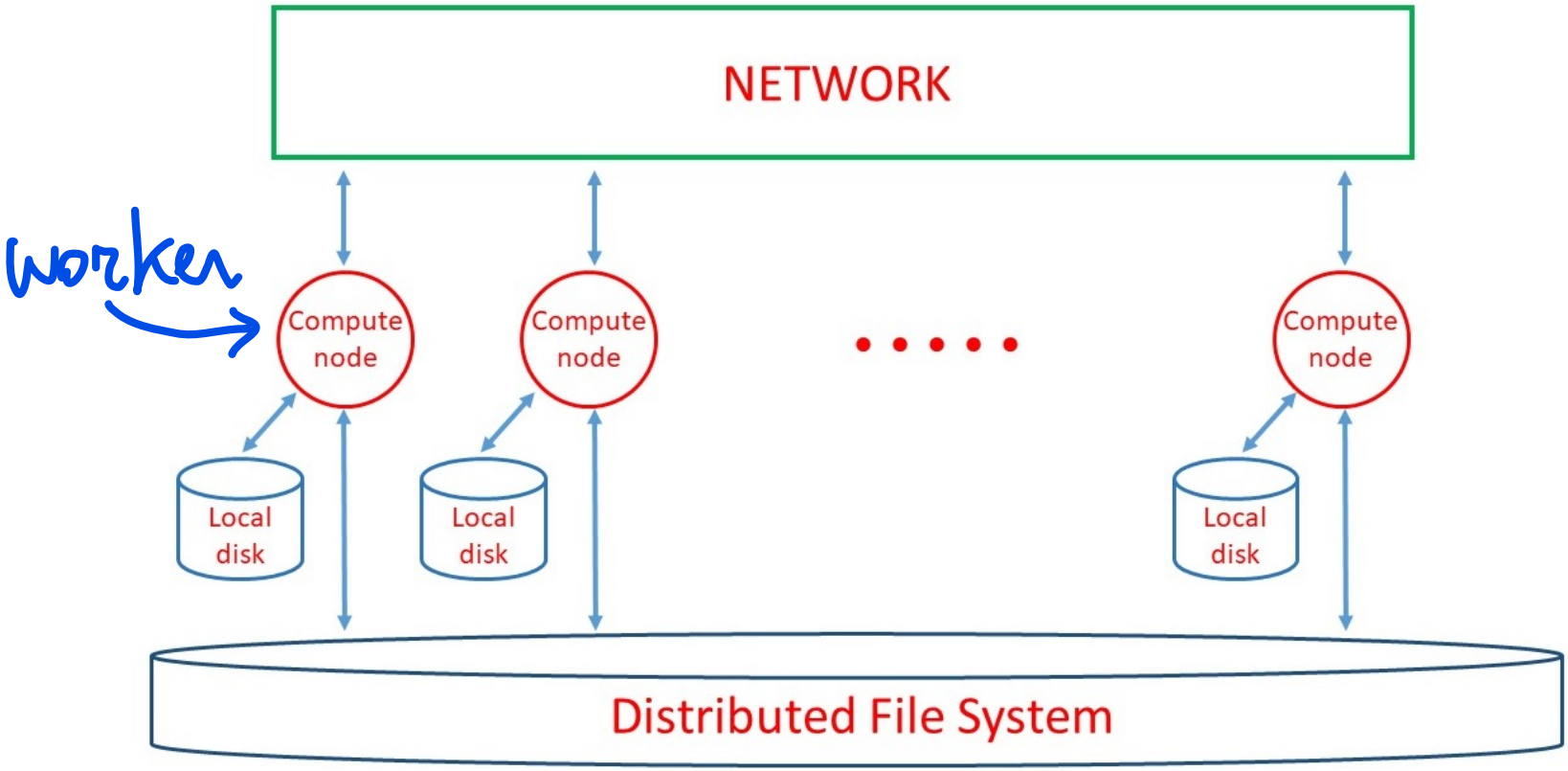
* Reduce phase: for each word w gather all $c_i(w)$'s and create the pairs $(w, c_1(w), \ldots, c_k(w))$

$$(w, c_1(w), \ldots, c_k(w)) \longrightarrow (w, \sum_i c_i(w))$$

reduce function

Obs. if $w \notin D_i$ then $c_i(w)$ does not exist

Implementation on a distributed system



The execution of a MapReduce computation is orchestrated among the available compute nodes, also called workers.

Implementation on a distributed system

The user program is forked into a master process and several worker processes. The master is in charge of assigning tasks to the workers and to monitor their status (idle, in-progress, completed). The workers execute the tasks and report their status to the master.

For each round:

- The applications of the map function to individual key-value pairs are grouped into map tasks. The applications of the reduce function to lists of values are grouped into reduce tasks.
- Input and output pairs reside on a Distributed File System, while intermediate pairs are stored on the workers' local disks.
- Data shuffle: between the map and reduce phases intermediate key-value pairs are moved from the workers where they were produced (by map tasks) to the workers where they must be processed (by reduce tasks).

  Obs.: *shuffle is often the most expensive operation of the round*

Map and reduce tasks

In a round, X map tasks and Y reduce tasks are created (X and Y are design parameters).

Map tasks:

- Input file is split into X chunks and each chunk forms the input of a map task.
- Each map task is assigned to a worker which applies the map function to each key-value pair of the corresponding chunk, and hashes the resulting intermediate key-values pairs into Y local files (intermediate files in [LRU14]) through a hash function h:

$$(k, v) \quad \rightarrow \quad \text{Local File } i = h(k) \bmod Y.$$

globally known

Observation: all intermediate pairs with the same key generated by the various map tasks are hashed into local files with the same index.

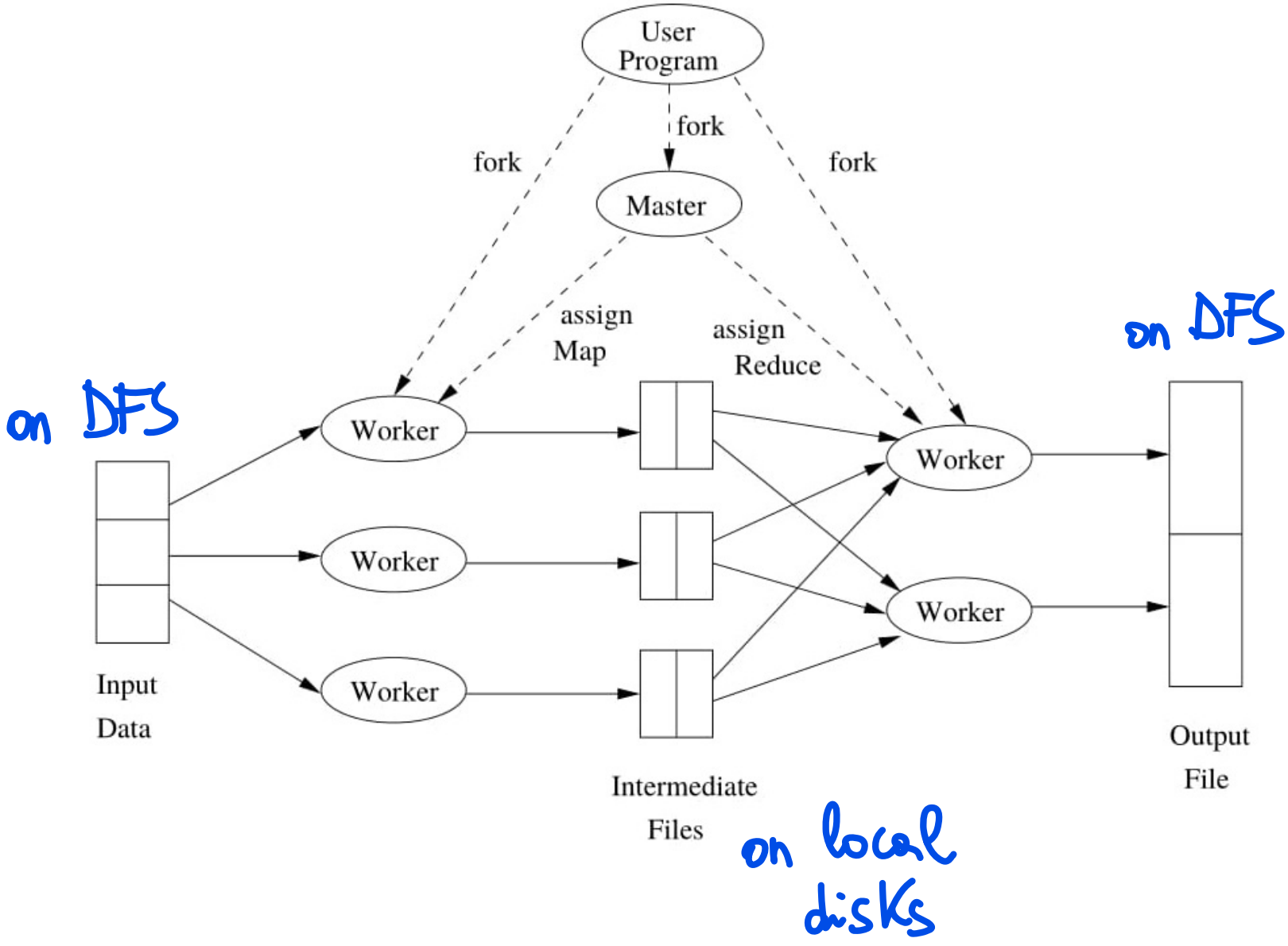
Map and reduce tasks

Reduce tasks:

- For each $0 \leq i < Y$, the key-value pairs residing in the local files of index i, transformed into (key, list-of-values) pairs form the input of the i-th reduce task.
- Each reduce task is assigned to a worker which applies the reduce function to each (key,list-of-values) pair included in the task and stored the output (key,value) pairs on the DFS.

TERMINOLOGY: The application of the reduce function to a (key, list-of-values) pair is called reducer

Figure 2.3 from [LRU14]



Dealing with faults

- The Distributed File System is fault-tolerant
- Master pings workers periodically to detect failures
- Worker failure:
  - Map tasks completed or in-progress at failed worker are reset to idle and will be rescheduled. Note that even if a map task is completed, the failure of the worker makes its output unavailable to reduce tasks, hence it must be rescheduled.
  - Reduce tasks in-progress at failed worker are reset to idle and will be rescheduled.
- Master failure: the whole MapReduce task is aborted

Analysis of a MapReduce algorithm

The analysis of an MR algorithm aims at estimating the following key performance indicators:

- Number of rounds R. Coarse measure of running time
- Local space M_L: maximum amount of main memory required, in any round, by *a single invocation* of the map or reduce function used in that round, for storing the input and any data structure needed by the invocation. relates to RAM available at compute nodes
- Aggregate space M_A: maximum amount of (disk) space which is occupied by the stored data at the beginning/end of the map or reduce phase of any round. relates to disk space available in the system

Observations:

- The indicators are usually estimated through asymptotic analysis (either worst-case or probabilistic) as functions of the input size.
- M_L bounds the amount of main memory required at each worker, while M_A bounds the overall amount of (disk) space that the executing platform must provide.

Analysis of Word count

$R = 1$

$M_L = ?$

N_i = # words in D_i (with repetitions)

k Documents $D_1 \ldots D_k$

Map Phase $O(\max\{N_i \cdot 1 \le i \le k\})$

Reduce Phase: $O(k)$

$\Rightarrow M_L = O(\max\{\max\{N_i \cdot 1 \le i \le k\}, k\}) =$

$= O(\underbrace{\max\{N_i \cdot 1 \le i \le k\}}_{\equiv N_{max}} + k) = O(N_{max} + k)$

Analysis of Word count

M_L not satisfactory for large N_{max} or k

$M_A = ?$

$|input| = O(N)$ $\quad N = \sum_{i=1}^{k} N_i$

$|intermediate\ pairs| = O(N)$

$|output| = O(N)$

$\Rightarrow M_A = O(N)$

Design goals for MapReduce algorithms

Here is a simple yet important observation.

Observation

For every problem solvable by a sequential algorithm in space S there exists a 1-round MapReduce algorithm with $M_L = M_A = \Theta(S)$: *run the sequential algorithm on the whole input with one reducer*.

Remark: the trivial solution implied by the observation is impractical for very large inputs for the following reasons:

- A platform with very large main memory is needed.
- No parallelism is exploited.

Design goals for MapReduce algorithms

In general, to process efficiently very large inputs on a distributed system, an algorithm should:

- break the computation into a small number of rounds;
- execute, in each round, several tasks in parallel;
- let each task work on a small subset of data;
- keep the costs of coordination and communication among the tasks under control.

1 Exploit parallelism ⇒ time efficiency

2 Limit the amount of main memory and disk space

Design goals for MapReduce algorithms

In MapReduce terms, the above design goals become:

Design Goals for MapReduce Algorithms

- Few rounds (e.g., $R = O(1)$);
- Sublinear local space (e.g., $M_L = O(|\text{input}|^{\epsilon})$, with $\epsilon < 1$);
- Linear aggregate space (i.e., $M_A = O(|\text{input}|)$), or only slightly superlinear;
- Low complexity of each map or reduce function.

Observation: algorithms usually exhibit tradeoffs between performance indicators: e.g., small $M_L \rightarrow$ large R.

Pros and cons of MapReduce

Why is MapReduce suitable for big data processing?

- Data-centric view. Algorithm design focus on data transformations targeting the above design goals.
- Usability. Programming frameworks (e.g., Spark) usually make allocation of tasks to workers, data management, handling of failures, totally transparent to the programmer.
- Portability/Adaptability. Applications can run on different platforms and the supporting framework will do best effort to exploit parallelism and minimize data movement costs.
- Cost. MapReduce applications can be run on moderately expensive platforms, and many popular cloud providers support their execution.

Pros and cons of MapReduce

What are the main drawbacks of MapReduce?

- Weakness of the number of rounds as performance measure. It ignores the runtimes of map and reduce functions and the actual volume of data shuffled. More sophisticated (yet, less usable) performance measures exist.
- Curse of the last reducer. In some cases, one or a few reducers may be much slower than the others, thus delaying the end of the round. Algorithm designers should aim at balancing the load (if at all possible) among reducers.

Basic Techniques

Partitioning

One of the main goals in the design of efficient MapReduce algorithms is to avoid that individual reducers process large amounts of data (e.g., sum of partial word counts from many documents). To achieve the goal, the relevant data should be partitioned, either deterministically or randomly, and the processing should be performed in stages.

We will see two examples:

- An improved version of Word count random partitioning
- A Class count primitive deterministic partitioning

Improved Word count

Consider the word count problem, k documents and N word occurrences overall, in a realistic scenario where k and N are huge (e.g., *huge number of small documents*).

If k is close to N then the 1-round algorithm needs $M_L = O(N)$

We now see how to reduce the local space requirements at the expense of an extra round.

Idea:

- Assign random keys in $[0, \sqrt{N}]$ to intermediate pairs so to create $\sqrt{N} = o(N)$ groups;
- Compute counts in two stages. (=> 2 rounds)

Round 1 : compute partial counts within each group with the same random key

Round 2 : Compute final counts aggregating partial ones

Improved Word count: algorithm

Input/output · as before

Round 1

Map Phase

key, value

$D_1 \to \{(x, (w, c_1(w)))$ · $w \in D_1$

x random int in $[0, \sqrt{N})$

$c_1(w) = \#$ occurrences of w in $D_1\}$

Reduce Phase · for each key $x \in [0, \sqrt{N})$ seperately gather all intermediate pairs $(x, (w, c_1(w)))$ and generate (x, L_x) where $L_x \equiv$ list of all $(w, c_i(w))$'s from intermediate pairs $(x, (w, c_1(w)))$

Improved Word count: algorithm

for each word w occurring in L_x generate the pair

$(w, c(x,w))$ where $c(x,w) = \sum_{(w, c_1(w)) \in L_x} c_1(w)$

$(w, c(x,w))$: output of R1 & input of R2

Round 2

Map Phase : empty

Reduce Phase : for each word w separately gather all $c(x,w)$'s $(x \in [0, \sqrt{N}) \Rightarrow \leq \sqrt{N}$ counts $c(x,w))$

$(w, \text{list of } c(x,w)\text{'s}) \longrightarrow (w, c(w) = \sum_{x \in [0,\sqrt{N})} c(x,w))$

OUTPUT

Improved Word count: analysis

Let:

- N_i = be the number of words in D_i.
- m_x = number of intermediate pairs with random key x.
- $m = \max\{m_x : 0 \leq x < \sqrt{N}\}$.

We have

- $R = 2$
- $M_L = O\left(\max\{N_i : 1 \leq i \leq k\} + m + \sqrt{N}\right)$.
- $M_A = O(N)$

$R = 2$ trivial

M_L

Round 1 Map phase: $O(\max\{N_i : 1 \le i \le k\})$

Reduce phase: $O(m)$

Round 2 Map phase: empty

Reduce phase: $O(\sqrt{N})$

$\Rightarrow M_L \in O(\max\{N_i : 1 \le i \le k\} + m + \sqrt{N})$

$M_A = O(N)$ the 2 rounds do not increase space requirements w.r.t. input

How large can m be?

Theorem

Suppose that the keys assigned to the intermediate pairs in Round 1 are i.i.d. random variables with uniform distribution in $[0, \sqrt{N})$. *Then, with probability at least* $1 - 1/N^5$

$$m = O\left(\sqrt{N}\right).$$

Therefore, from the theorem and the preceding analysis we get

$$M_L = O\left(\max\{N_i : 1 \leq i \leq k\} + \sqrt{N}\right), \approx O(\sqrt{N}) \text{ if documents are small}$$

with probability at least $1 - 1/N^5$. In fact, for large N the probability becomes *very close to 1*.

N.B.: This is an example of probabilistic analysis, as opposed to more traditional worst-case analysis.

Proof of theorem

We need the following very useful technical tools.

Union bound

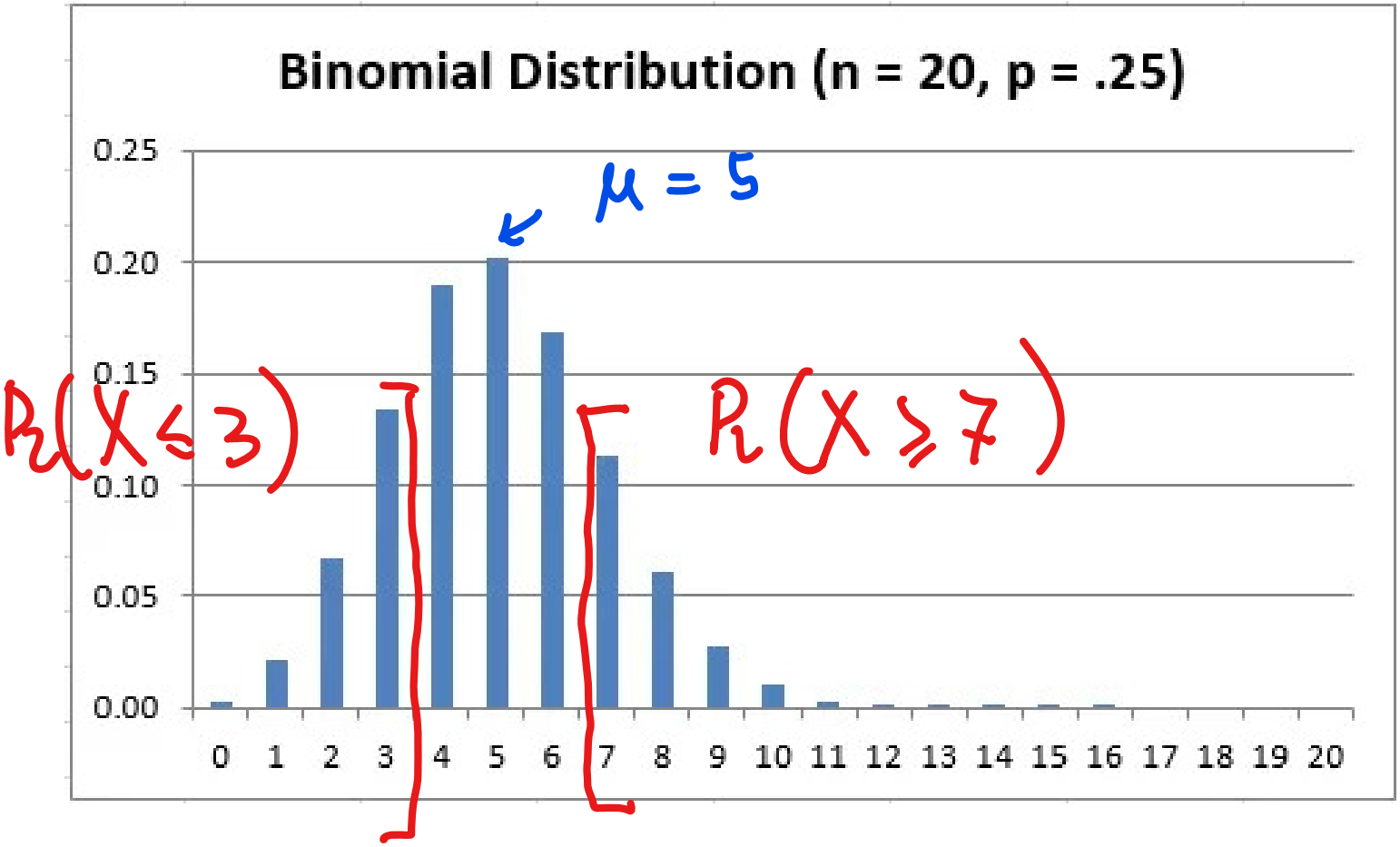
Given a countable set of events $E_1, E_2, \ldots, E_r$, we have:

$$\Pr\left(\bigcup_{i=1}^{r} E_i\right) \leq \sum_{i=1}^{r} \Pr(E_i).$$

Chernoff bound (see proof in [MU05])

Let $X_1, X_2, \ldots, X_n$ be n i.i.d. Bernoulli random variables, with $\Pr(X_i = 1) = p$, for each $1 \leq i \leq n$. Thus, $X = \sum_{i=1}^{n} X_i$ is a Binomial(n, p) random variable. Let $\mu = E[X] = n \cdot p$. For every $\delta_1 \geq 5$ and $\delta_2 \in (0, 1)$ we have that

$$\begin{aligned} \Pr(X \geq (1+\delta_1)\mu) &\leq 2^{-(1+\delta_1)\mu} \\ \Pr(X \leq (1-\delta_2)\mu) &\leq 2^{-\mu\delta_2^2/2} \end{aligned}$$


Binomial Distribution (n = 20, p = .25)
0.25
0.20
0.15
0.10
0.05
0.00
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
μ = 5
P(X ≤ 3)
P(X ≥ 7)

Proof of theorem

N' = # intermediate pairs produced by Map Phase in R1 $\Rightarrow$ $N' \leq N$

Fix a key $x \in [0, \sqrt{N})$ arbitrarily

$m_x \equiv \text{Binomial}(N', 1/\sqrt{N})$ (Hp $\sqrt{N}$ integer)

$\mu = E[m_x] = N' \cdot 1/\sqrt{N} \leq \sqrt{N}$

Observe that $6\sqrt{N} \geq 6\mu = (1+5)\mu$

$\Rightarrow 6\sqrt{N} = (1+\delta_1)\mu \quad \delta_1 \geq 5$

Proof of theorem

By Chernoff Bound

$$P_r(m_x \geq 6\sqrt{N}) = P_r(m_x \geq (1+\delta_1)\mu) \leq 2^{-(1+\delta_1)\mu}$$

$$2^{-(1+\delta_1)\mu} = 2^{-6\sqrt{N}} \leq 2^{-6\log_2 N} = (1/N)^6$$

Obs. For $N \geq 2$ $\sqrt{N} \geq \log_2 N$

Now $P_r\left(m = \max_{x \in [0,\sqrt{N})} m_x \geq 6\sqrt{N}\right) =$

$$P_r(\exists x \in [0,\sqrt{N}) \text{ s.t } m_x \geq 6\sqrt{N})$$

Proof of theorem

By union bound $P_2(\exists x \in [0, \sqrt{N}) \text{ s.t. } m_x \geq 6\sqrt{N})$

$\leq \sum_{x \in [0, \sqrt{N})} P_2(m_x \geq 6\sqrt{N}) \leq \sqrt{N} \frac{1}{N^6} \leq \frac{1}{N^5}$

$\Rightarrow$ With probability $\geq 1 - \frac{1}{N^5}$ all m_x's are $< 6\sqrt{N}$

$\Rightarrow m = O(\sqrt{N})$

□

Observations

- The choice of partitioning the intermediate pairs into $O\left(\sqrt{N}\right)$ groups provides an example of a simple tradeoff between local space and number of rounds.
- We will often target local space around $O\left(\sqrt{|\text{input size}|}\right)$, since it is "sufficiently sublinear" bound and can be often achieved with few rounds.
- However, in many cases the approach can be generalized to attain a given target bound on M_L, trading rounds for space. One of the exercises will explore this issue.

Class count

Problem: given a set S of N objects with class labels, count how many objects belong to each class.

More precisely:

Input: Set S of N objects represented by pairs $(i, (o_i, \gamma_i))$, for $0 \leq i < N$, where o_i is the i-th object, and γ_i its class.
Output: The set of pairs $(\gamma, c(\gamma))$ where γ is a class labeling some object of S and $c(\gamma)$ is the number of objects of S labeled with γ.

Class count in 1 round (straighforward)

Round 1

Map Phase $(i, (o_i, \gamma_i)) \longrightarrow (\gamma_i, 1)$

Reduce Phase : for each class γ separately gether all pairs $(\gamma, 1)$ and create (γ, L_γ) $L_\gamma \equiv$ list of 1's from pairs $(\gamma, 1)$

$(\gamma, L_\gamma) \longrightarrow (\gamma, c(\gamma) = |L_\gamma|)$

output pair

Class count in 1 round (straighforward)

$R = 1$

$M_L = ?$ Map Phase $\cdot\ O(1)$ ← Hp each input pair takes constant space

Reduce Phase $\cdot\ O(\max_\gamma c(\gamma))$

$\Rightarrow M_L = O(\max_\gamma c(\gamma))$

not satisfactory since, in the worst case,

$\max_\gamma c(\gamma) = \Theta(N)$, hence $M_L = \Theta(N)$

$M_A = \Theta(N)$

Class count in 2-rounds

Round 1 : Compute partial counts

Map Phase

$(i, (o_i, \gamma_i)) \longrightarrow (i \bmod \sqrt{N}, \gamma_i)$

($i \bmod \sqrt{N}$: remainder of integer division $i/\sqrt{N}$)

Reduce Phase: for each k, $k \in [0, \sqrt{N}-1]$

separately gather all intermediate pairs (k, γ) and create (k, L_k) where L_k = list of classes γ in pairs (k, γ) (obs a class can occur multiple times in L_k)

$(k, L_k) \longrightarrow \{(\gamma, c(k,\gamma)) \cdot \gamma \in L_k$ and $c(k,\gamma) = \#$occurrences of γ in $L_k\}$

Class count in 2 rounds

Round 2

Map Phase: empty

Reduce Phase: for each class γ separately gether all pairs $(\gamma, c(k,\gamma))$ $k \in [0, \sqrt{N}-1]$ and create $(\gamma, \text{list of } c(k,\gamma)\text{'s})$ $\leq \sqrt{N}$ elements in the list

$(\gamma, \text{list of } c(k,\gamma)\text{'s}) \rightarrow (\gamma, c(\gamma) = \sum_{c(k,\gamma) \text{ in list}} c(k,\gamma))$

OUTPUT PAIR

Analysis

$R = 2 \quad M_L = O(\sqrt{N}) \quad M_A = O(N)$ ← EXERCISE

Trading accuracy for efficiency

There are problems for which exact algorithms may be too space/time-consuming, hence impractical for very large inputs.

In these scenarios, settling for approximate solutions (if acceptable for the application) may greatly improve efficiency.

We'll now see an example through an important primitive for the processing of pointsets from metric spaces (e.g., spatial datasets).

Maximum pairwise distance ≡ DIAMETER

Problem: given a set S of N points from some metric space (e.g., $\Re^3$) determine the maximum distance between two points.

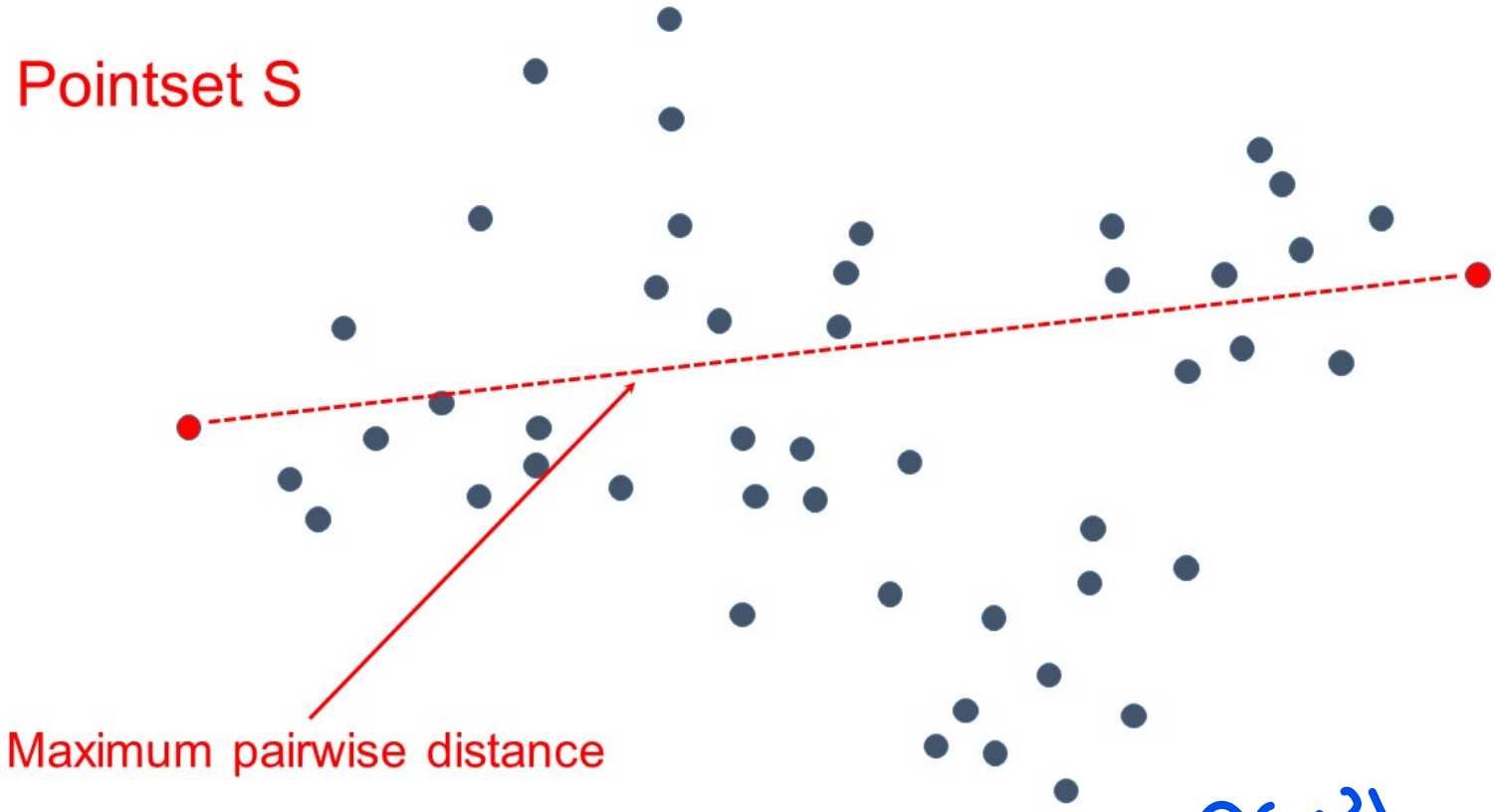
More precisely: let $d(\cdot, \cdot)$ be the distance function

Input: Set S of N points represented by pairs (i, x_i), for $0 \leq i < N$, where x_i is the i-th point.
Output: $(0, \max_{0 \leq i,j < N} d(x_i, x_j))$.

Exercise

Design an MR algorithm for the problem which requires $R = O(1)$, $M_L = O\left(\sqrt{N}\right)$ and $M_A = O\left(N^2\right)$.

Maximum pairwise distance



Sequential setting: Trivial algorithm: $\Theta(N^2)$ complexity
Euclidean points: $O(N \lg N)$ "
Points from arbitrary spaces: ? $\approx \Theta(N^2)$ "

Maximum pairwise distance

We can improve the aggregate space if we tolerate a factor 2 error in the estimate of $d_{\max}$.

For an arbitrary $x_i \in S$ define

$$d_{\max}(i) = \max\{d(x_i, x_j) \ : \ 0 \leq j < N\}.$$

↳ max distance from x_i

Lemma

For any $0 \leq i < N$ *we have* $d_{\max} \in [d_{\max}(i), 2d_{\max}(i)]$.

Maximum pairwise distance

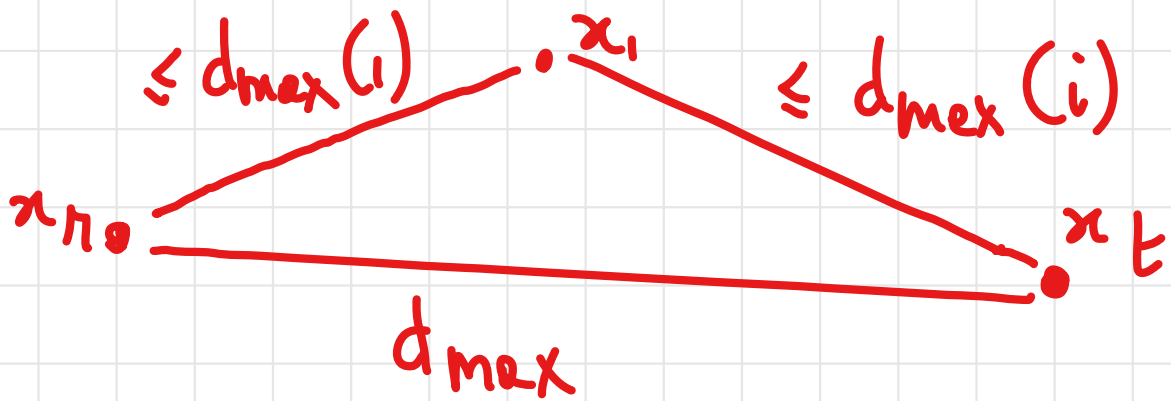
Proof of Lemma:

$d_{max} \geq d_{max}(i)$ trivial

$d_{max} \leq 2\, d_{max}(i)$

Suppose $d_{max} = d(x_r, x_t)$. By triangle inequality

$d(x_r, x_t) \leq d(x_r, x_i) + d(x_i, x_t) \leq 2\, d_{max}(x_i)$



Maximum pairwise distance

Let's compute $d_{max}(0)$ 1 Round ? No $M_L = \Theta(N)$

2-Round algorithm

Round 1

Map Phase

$i=0 \quad (0, x_0) \longrightarrow (j, (0, x_0)) \quad 0 \le j < \sqrt{N}$

$i>0 \quad (i, x_i) \longrightarrow (i \bmod \sqrt{N}, (i, x_i))$

Reduce Phase for each key $j \in [0, \sqrt{N})$ create

(j, L_j) $L_j \equiv$ list of (i, x_i) from intermediate pairs with key j Obs. $(0, x_0) \in L_j$

Maximum pairwise distance

$(j, L_j) \rightarrow (j, d_{max}(o_j))$ (o, x_o) is included in L_j

$$d_{max}(o_j) = \max_{(i, x_i) \in L_j} d(x_o, x_i)$$

Round 2

Map Phase: $(j, d_{max}(o_j)) \rightarrow (0, d_{max}(o, j))$

Reduce Phase: gather all pairs $(0, d_{max}(o, j))$ and create $(0, L)$ $L \equiv$ list of all $d_{max}(o_j)$'s, and return $(0, d_{max}(o) = \max\{d_{max}(o, j) : 0 \le j < \sqrt{N}\})$

$M_A = O(N + \sqrt{N}) = O(N)$ $M_L = O(\sqrt{N})$

Exercise

For each of the 4 problems listed in Exercise 2.3.1 of [LRU14], design an $O(1)$-round MR algorithm which uses $O\left(\sqrt{N}\right)$ local space and linear aggregate space, where N is the input size.

Exercise

Consider the Class count problem with the input consisting only of object-class pairs (o, γ), i.e., where no integer keys in $[0, N)$ are provided (now the objects act as keys). Devise an efficient MR algorithm for the problem.

Hint: use random keys and target probabilistic guarantees.

Exercise

Design and analyze an efficient MR algorithm for approximating the maximum pairwise distance, which uses local space $M_L = O\left(N^{1/4}\right)$. Can you generalize the algorithm to attain $M_L = O\left(N^{\epsilon}\right)$, for any $\epsilon \in (0, 1/2)$?

Exercise

Design an $O(1)$-round MR algorithm for computing a matrix-vector product $W = A \cdot V$, where A is an $m \times n$ matrix, V is an n-vector, and $m \leq \sqrt{n}$. Your algorithm must use $o(n)$ local space and linear (i.e., $O(mn)$) aggregate space.

How would your solution change if m were larger?

Summary

- MapReduce.
  - Motivating scenario.
  - Main Features, platforms and software frameworks.
  - MapReduce computation: high-level structure, algorithm specification, implementation on a distributed system, analysis, key peformance indicators.
  - Algorithm design goals.
- Basic Techniques and Primitives.
  - Chernoff and union bounds.
  - Partitioning: randomized (Word count 2) and deterministic (Class count).
  - Efficiency-accuracy tradeoffs: Maximum pariwise distance

References

- **[LRU14]** J. Leskovec, A. Rajaraman and J. Ullman. Mining Massive Datasets. Cambridge University Press, 2014. Chapter 2 and Section 6.4
- **[DG08]** J. Dean and A. Ghemawat. MapReduce: simplified data processing on large clusters. OSDI'04 and CACM 51,1:107113, 2008
- **[MU05]** M. Mitzenmacher and E. Upfal. Proability and Computing: Randomized Algorithms and Probabilistic Analysis. Cambridge University Press, 2005. (Chernoff bounds: Theorems 4.4 and 4.5)
- **[P+12]** A. Pietracaprina, G. Pucci, M. Riondato, F. Silvestri, E. Upfal: Space-round tradeoffs for MapReduce computations. ACM ICS'112.